BCS703 – MODULE 5

DKIM • IPsec: applications & benefits • Inbound / outbound packets • Transport & tunnel mode • IKE header format & algorithm

#	Topic
1	DomainKeys Identified Mail (DKIM)
2	IP Security – general applications and benefits
3	Transport mode and Tunnel mode (with ESP)
4	IP security policy – SA, SAD, SPD
5	Outbound packets and Inbound packets (IP traffic processing)
6	Internet Key Exchange (IKE) – key determination algorithm
7	IKE header format, payloads and IKEv2 exchanges
8	Last-minute revision sheet

How to use: every topic has (1) a quick definition, (2) the steps/points in the order the textbook gives them, (3) the textbook figure, and (4) an **Exam keywords** box. Write the bold words in your answer to score marks. Content and figures are taken only from your uploaded textbook pages.

1. DOMAINKEYS IDENTIFIED MAIL – DKIM (Section 19.9)

DKIM is a specification for **cryptographically signing email messages**, permitting a **signing domain to claim responsibility** for a message in the mail stream. Recipients (or agents acting for them) **verify the signature by querying the signer's domain directly to retrieve the public key**, and so confirm that the message was attested to by a party in possession of the **private key of the signing domain**. DKIM is an **Internet Standard (RFC 6376)**, widely adopted (corporations, government agencies, gmail, Yahoo!, many ISPs).

1.1 Email threats (RFC 4686 – threats motivating DKIM)

Characteristics – three levels of attackers

1. **Low end:** want to send email a recipient does not want; use commercial tools to **falsify the origin address**, making spam filtering by address/domain difficult.
2. **Professional bulk spam senders:** commercial enterprises sending for third parties; use **MTAs, registered domains, networks of compromised computers (zombies)**, harvest addresses.
3. **Most sophisticated, financially motivated** (e.g., email-based fraud): use all the above and may **attack Internet infrastructure** – **DNS cache poisoning** and **IP routing attacks**.

Capabilities of an attacker (RFC 4686 – 11 points)

1. Submit messages to MTAs and **MSAs** at multiple locations.
2. Construct **arbitrary header fields**, including claiming to be mailing lists, resenders, other agents.
3. **Sign messages** on behalf of domains under their control.
4. Generate large numbers of unsigned/apparently signed messages for **denial of service**.
5. **Resend** messages previously signed by the domain.
6. Transmit messages using any **envelope information**.
7. Act as authorized submitter from a **compromised computer**.
8. **Manipulate IP routing**.
9. Limited influence over **DNS** (cache poisoning).
10. Access to significant computing resources (zombie computers) for brute force.
11. Ability to **eavesdrop** on existing traffic (e.g., wireless).

Location

DKIM focuses on attackers **outside the administrative units** of the claimed originator and recipient, where trust relationships for authenticated submission do not exist/scale. Inside those units, other mechanisms (authenticated message submission) are easier. External attackers exploit the **“any-to-any” nature of email**; they may send unsigned, wrongly signed, or correctly signed messages from domains with little traceability, or pose as mailing lists or greeting cards.

1.2 DKIM strategy

DKIM provides an email authentication technique **transparent to the end user**. A user's message is **signed with the private key of the administrative domain** from which it originates. The signature covers **all of the message content and some of the RFC 5322 headers**. At the receiving end the **MDA** gets the public key via **DNS** and verifies the signature, so mail that claims a domain but originates elsewhere **fails and can be rejected**. (S/MIME and PGP instead use the **originator's** private key.) Motivation:

1. S/MIME needs **both sender and receiver** to use S/MIME; most mail does not.
2. S/MIME signs **only message content**, so RFC 5322 header (origin) information can be compromised.
3. DKIM is **not implemented in client programs (MUAs)** → transparent; user takes no action.
4. DKIM applies to **all mail from cooperating domains**.
5. DKIM lets **good senders prove** they sent a message and prevents forgers **masquerading** as good senders.

1.3 Simple deployment (Fig 19.10)

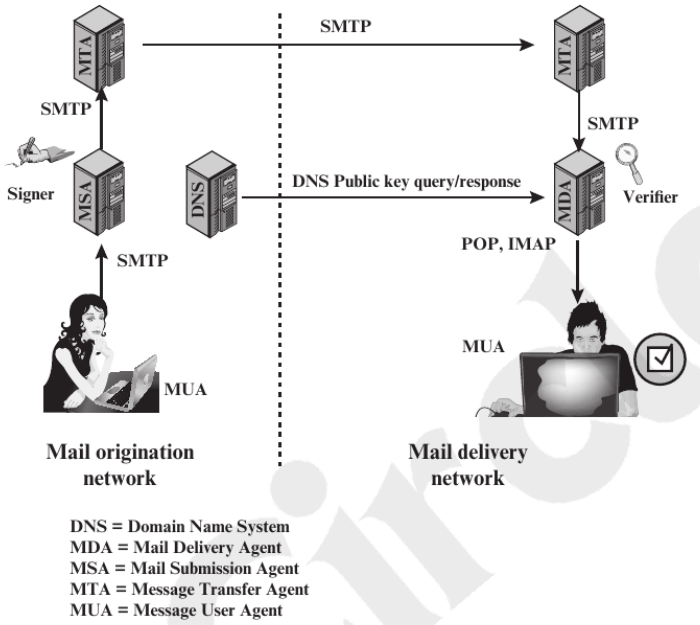


Figure 19.10 Simple Example of DKIM Deployment

Fig 19.10 Simple Example of DKIM Deployment

- User's email client generates a message → into the MHS to an **MSA** in the user's administrative domain.
- Content + selected **RFC 5322 headers** are **signed by the email provider using the provider's private key** (signer = corporate network, ISP or public email facility such as gmail).
- Signed message passes through the Internet via a sequence of **MTAs**.
- At destination the **MDA** retrieves the **public key** (DNS query/response) and **verifies** the signature before passing the message to the email client.
- Default algorithm: **RSA with SHA-256** (RSA with SHA-1 also allowed).

1.4 DKIM functional flow (Fig 19.11)

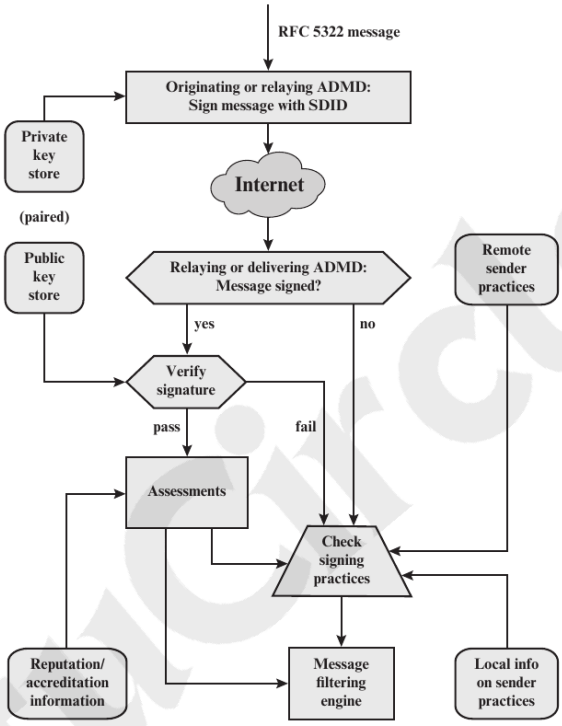


Figure 19.11 DKIM Functional Flow

Fig 19.11 DKIM Functional Flow

- Processing is divided between a **signing ADMD** (Administrative Management Domain) and a **verifying ADMD**.
- **Signing** is done by an authorized module using the **private key store**; in the originating ADMD it may be done by the **MUA, MSA or an MTA**. Message is signed with the **SDID** (signing domain identifier).

- **Verifying** is done by an authorized module in the verifying ADMD (MTA, MDA or MUA), using the **public key store**. It verifies the signature or determines whether a signature was required.
- Signature **passes** → **reputation/accreditation information** assesses the signer → passed to the **message filtering engine**.
- Signature **fails** or **no signature** from the author's domain → **signing practices** of the author are retrieved (remote/local sender-practice info) and passed to the filtering engine. E.g., if the sender (gmail) uses DKIM but no signature is present, the message may be considered **fraudulent**.

1.5 DKIM-Signature header

The signature is inserted into the RFC 5322 message as an additional header starting with **Dkim-Signature**. Example from the textbook:

```
Dkim-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed;
   d=gmail.com; s=gamma; h=domainkey-signature:mime-version:received:date:
   message-id:subject:from:to:content-type:content-transfer-encoding;
   bh=5mZvQDyCRuyLb1Y28K4zgS2MPOemFToDBgvbJ7GO90s=;
   b=PcUvPSDygb4ya5Dyj1rbZGp/VyRiScuaz7TTGJ5qW5slM+klzv6kcfYdGDH...
```

Tag	Meaning (textbook)
v=	DKIM version
a=	Algorithm used to generate the signature; must be **rsa-sha1** or **rsa-sha256**
c=	Canonicalization method used on header and body
d=	Domain name identifying the responsible person/organization = **Signing Domain IDentifier (SDID)** (in the example: gmail)
s=	**Selector** – a name associated with a key; lets different keys be used for the same domain (expire old keys, departmental signing); verifier uses it to retrieve the proper key
h=	**Signed header fields** – colon-separated list of header names presented to the signing algorithm
bh=	**Hash of the canonicalized body**; helps diagnose verification failures
b=	**Signature data** in base64 = the **encrypted hash code**

- **Canonicalization** is performed on header and body before signing, to cope with minor changes en route: character encoding, trailing white space, **folding/unfolding** of header lines. The message itself is not changed, so the verifier repeats canonicalization. Two header algorithms and two body algorithms: **"simple"** (tolerates almost no modification) and **"relaxed"** (tolerates common modifications).

Related (Section 19.10) – DMARC (RFC 7489): lets email senders specify policy on how their mail should be handled and what reports receivers send back. Works with SPF and DKIM: DKIM/SPF do not tell receivers whether they are in use and give no feedback to senders. DMARC requires the **RFC 5322 From domain to be aligned** with an authenticated identifier – for DKIM, the **DKIM signing domain**; for SPF, the **SPF-authenticated domain**.

Exam keywords – DKIM
RFC 6376 • cryptographically signing • signing domain • private key of domain • public key via DNS • transparent to user • RFC 5322 headers • MSA signs, MDA verifies • RSA-SHA256 • Dkim-Signature • v a c d s h bh b • SDID • selector • canonicalization simple/relaxed • ADMD • reputation • message filtering engine • RFC 4686 threats • zombies • DNS cache poisoning

2. IP SECURITY – OVERVIEW, APPLICATIONS AND BENEFITS (Section 20.1)

- **1994:** Internet Architecture Board (IAB) report **"Security in the Internet Architecture" (RFC 1636)** identified the need to secure the network infrastructure from **unauthorized monitoring and control of network traffic** and to secure **end-user-to-end-user traffic** using **authentication and encryption**.
- IAB included authentication and encryption as necessary features of **IPv6**; the capabilities are usable with both **IPv4 and IPv6**.
- Application-specific mechanisms exist (S/MIME, PGP, Kerberos, SSL), but users have concerns across layers. Security at the **IP level** secures even **security-ignorant applications**.
- **IP-level security has three functional areas: authentication** (packet really came from the claimed source and was not altered), **confidentiality** (encryption to prevent eavesdropping) and **key management** (secure exchange of keys).

2.1 Applications of IPsec

IPsec secures communications across a **LAN, private and public WANs, and the Internet**. Examples:

Application	Explanation (textbook)
Secure branch office connectivity over the Internet	Company builds a secure **virtual private network (VPN)** over the Internet/public WAN; reduces need for private networks, saving cost and network-management overhead.
Secure remote access over the Internet	End user with IP security protocols makes a **local call to an ISP** and gains secure access to the company network; reduces toll charges for travelling employees and telecommuters.
Extranet and intranet connectivity with partners	Secures communication with other organizations, ensuring **authentication and confidentiality** and providing a **key-exchange mechanism**.
Enhancing electronic commerce security	Even if Web/e-commerce apps have built-in security, IPsec guarantees all traffic designated by the network administrator is **encrypted and authenticated** – an additional layer.

Principal feature: IPsec can **encrypt and/or authenticate all traffic at the IP level**, so all distributed applications (remote logon, client/server, email, file transfer, Web access) can be secured.

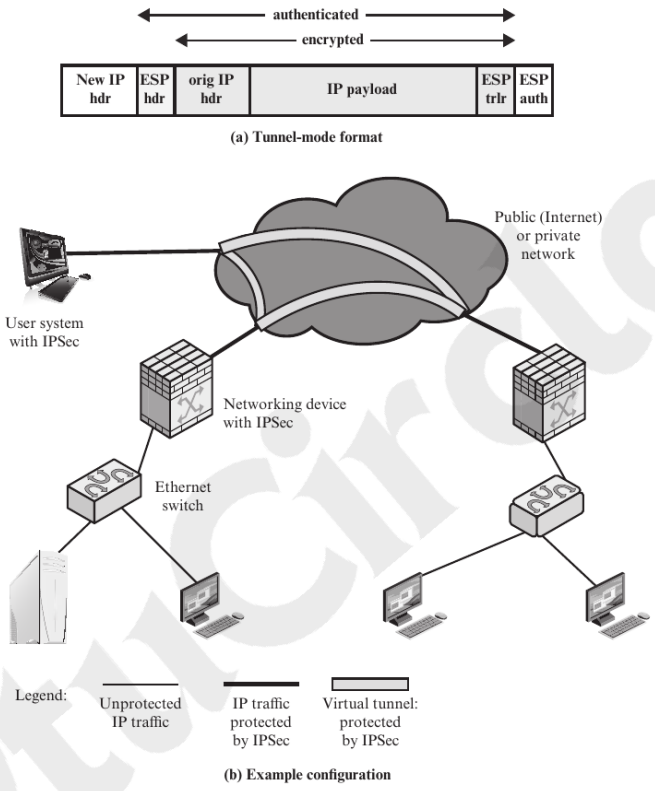


Figure 20.1 An IPSec VPN Scenario

Fig 20.1 An IPsec VPN Scenario – (a) tunnel-mode format, (b) example configuration

- An organization has LANs at dispersed locations; **non-secure IP traffic** on each LAN. For off-site traffic over a private/public WAN, IPsec protocols run in a **networking device (router or firewall)** at the edge of each LAN. This device **encrypts all traffic going into the WAN and decrypts traffic coming from it** – **transparent** to workstations and servers on the LAN. Individual users who dial in must implement IPsec on their workstations.
- For VPNs both authentication and encryption are desired: (1) unauthorized users must not penetrate the VPN; (2) eavesdroppers on the Internet must not read messages.

2.2 Benefits of IPsec

1. In a **firewall or router** it provides **strong security for all traffic crossing the perimeter**; traffic within a company/workgroup does **not incur** security-processing overhead.
2. In a firewall it is **resistant to bypass** if all outside traffic must use IP and the firewall is the only entrance from the Internet.
3. It is **below the transport layer (TCP, UDP)**, hence **transparent to applications** – no software change on user/server systems (even when implemented in end systems, upper-layer software is unaffected).
4. It can be **transparent to end users** – no training, no issuing/revoking keying material per user.

5. It can provide security for **individual users** if needed (off-site workers; secure virtual subnetwork for sensitive applications).

2.3 Routing applications

IPsec can assure that: a **router advertisement** comes from an authorized router; a **neighbor advertisement** comes from an authorized router; a **redirect message** comes from the router to which the initial packet was sent; a **routing update is not forged**. Without this an opponent can disrupt communications or divert traffic. Routing protocols such as **OSPF** should run on top of IPsec security associations between routers.

2.4 IPsec documents and services

Group	Description
Architecture	General concepts, requirements, definitions, mechanisms – **RFC 4301**
Authentication Header (AH)	Extension header for message authentication – **RFC 4302**; **deprecated** (ESP provides authentication), kept for backward compatibility
Encapsulating Security Payload (ESP)	Encapsulating header and trailer for encryption or combined encryption/authentication – **RFC 4303**
Internet Key Exchange (IKE)	Key management – main spec **RFC 7296 (IKEv2)**
Cryptographic algorithms	Encryption, message authentication, PRFs, key exchange
Other	Security policy, MIB content

IPsec services (RFC 4301): access control; connectionless integrity; data origin authentication; rejection of replayed packets (partial sequence integrity); confidentiality (encryption); limited traffic-flow confidentiality. The two protocols: **AH** (authentication) and **ESP** (combined encryption/authentication).

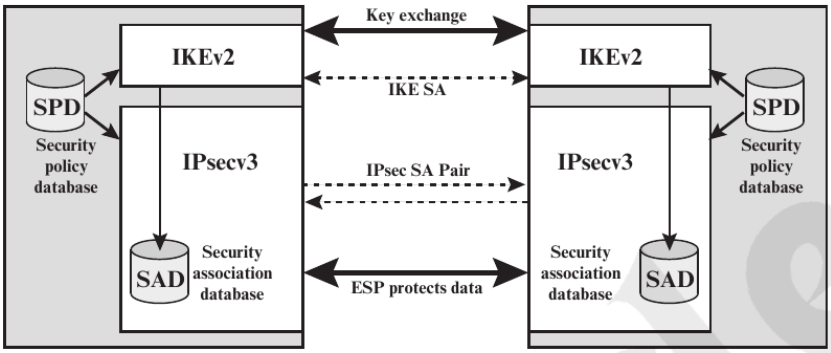


Figure 20.2 IPsec Architecture

Fig 20.2 IPsec Architecture

Exam keywords – IPsec applications & benefits
RFC 1636 • IAB • IPv4 & IPv6 • authentication • confidentiality • key management • VPN • branch office • remote access • extranet/intranet • e-commerce • below transport layer • transparent • firewall/router • bypass-resistant • routing (OSPF) • AH deprecated • ESP • RFC 4301 • IKEv2 RFC 7296

3. TRANSPORT MODE AND TUNNEL MODE (Sections 20.1 and 20.3)

Both **AH and ESP** support two modes of use: **transport mode** and **tunnel mode**.

3.1 Transport mode

- Provides protection **primarily for upper-layer protocols** – protection extends to the **payload of an IP packet** (e.g., TCP or UDP segment, ICMP packet).
- Typically used for **end-to-end communication between two hosts** (client–server, two workstations).
- IPv4: payload = data that normally follows the IP header. IPv6: payload = data after the IP header and extension headers (except possibly destination options header).
- **ESP in transport mode encrypts and optionally authenticates the IP payload but not the IP header.** AH in transport mode authenticates the IP payload and selected portions of the IP header.

Operation of transport mode ESP

1. At the source, the block of data (**ESP trailer + entire transport-layer segment**) is encrypted and the plaintext replaced by ciphertext to form the IP packet; authentication added if selected.
2. Packet is routed; each intermediate router examines and processes the **IP header + any plaintext IP extension headers** but **not the ciphertext**.
3. Destination examines the IP header, then on the basis of the **SPI** in the ESP header decrypts the rest to recover the plaintext transport-layer segment.

- **Advantage:** confidentiality for **any application** that uses it, without each application implementing it. **Drawback: traffic analysis** is possible on the transmitted packets.
- IPv4: ESP header is inserted **immediately before the transport-layer header**; ESP trailer (Padding, Pad Length, Next Header) placed **after the IP packet**; ESP authentication data added after trailer. **Encryption covers** transport segment + ESP trailer; **authentication covers** ciphertext + ESP header.
- IPv6: ESP is treated as an **end-to-end payload** (not processed by intermediate routers); ESP header appears after IPv6 base header and hop-by-hop, routing and fragment extension headers.

3.2 Tunnel mode

- Provides protection to the **entire IP packet**. After AH/ESP fields are added, the **entire packet plus security fields is treated as the payload of a new outer IP packet with a new outer IP header**.
- The original (inner) packet travels through a **tunnel**; **no routers on the way can examine the inner IP header**. The new packet may have totally different source and destination addresses → added security.
- Used when **one or both ends of an SA are a security gateway** (firewall or router implementing IPsec). Hosts behind firewalls can communicate securely **without implementing IPsec**; their unprotected packets are tunneled through external networks by tunnel-mode SAs set up by the gateway.
- **ESP in tunnel mode encrypts and optionally authenticates the entire inner IP packet, including the inner IP header.** AH in tunnel mode authenticates the entire inner packet and selected portions of the outer IP header.
- Useful for **VPNs**; **counters traffic analysis**; relieves internal hosts of encryption burden; **simplifies key distribution** (fewer keys).

Example – Host A to Host B through firewalls

1. Host A generates an IP packet with destination = host B and routes it to the **firewall/secure router** at the boundary of A's network.
2. The firewall **filters all outgoing packets** to determine if IPsec processing is needed. If yes, it performs IPsec processing and **encapsulates the packet with an outer IP header** (source = this firewall, destination = B's firewall).
3. The packet is routed to B's firewall; **intermediate routers examine only the outer IP header**.
4. At B's firewall the **outer header is stripped**, the inner packet is decrypted and delivered to B.

Tunnel-mode ESP: external host to internal host (4 steps)

1. Source prepares an inner IP packet (destination = target internal host), **prefixes an ESP header**, encrypts packet + ESP trailer, may add authentication data, then encapsulates with a **new IP header whose destination is the firewall** = outer IP packet.
2. Outer packet routed to the destination firewall; intermediate routers examine only the outer header (and outer extension headers), **not the ciphertext**.
3. Destination firewall processes the outer header, uses the **SPI** in the ESP header to decrypt the remainder and recover the **plaintext inner IP packet**, which is transmitted in the internal network.
4. Inner packet is routed through zero or more routers to the destination host.

3.3 Comparison and diagrams

	Transport Mode SA	Tunnel Mode SA
AH	Authenticates IP payload and selected portions of IP header and IPv6 extension headers.	Authenticates entire inner IP packet (inner header + payload) plus selected portions of outer IP header and outer IPv6 extension headers.
ESP	Encrypts IP payload and any IPv6 extension headers following the ESP header.	Encrypts entire inner IP packet.
ESP with Authentication	Encrypts IP payload and IPv6 extension headers following ESP header. Authenticates IP payload but **not IP header**.	Encrypts entire inner IP packet. Authenticates inner IP packet.

Point	Transport mode	Tunnel mode
Protects	Payload (upper-layer data)	Entire IP packet
IP header	Original header remains, **not protected** by ESP	Inner header protected; **new outer header** added
Typical use	**Host-to-host** (end-to-end)	**Gateway-to-gateway** / host-to-gateway (VPN)
Hosts need IPsec?	Yes	No (gateway does it)
Traffic analysis	Possible	Countered
Overhead	Smaller	Larger (extra IP header)

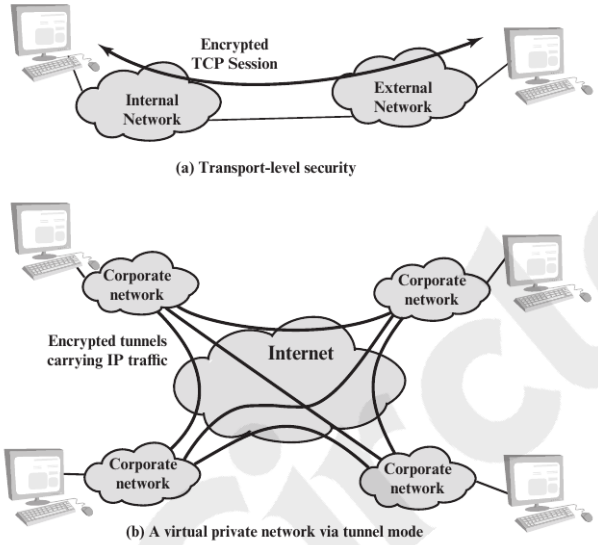


Figure 20.7 Transport-Mode versus Tunnel-Mode Encryptionx

Fig 20.7 Transport-Mode versus Tunnel-Mode Encryption (VPN via tunnel mode)

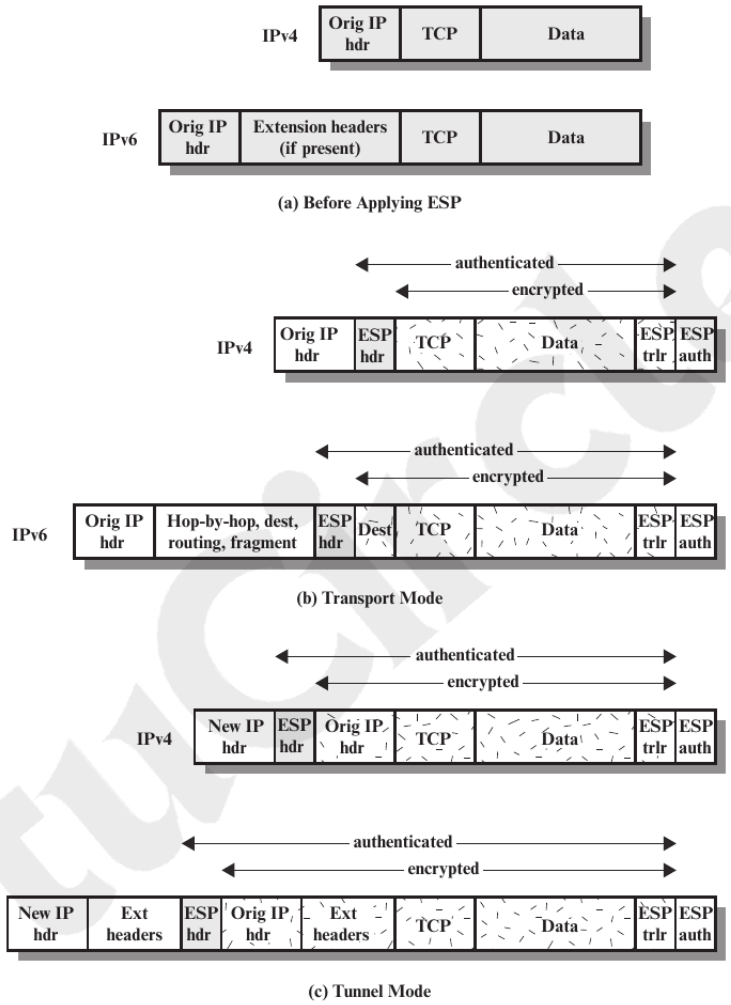


Figure 20.8 Scope of ESP Encryption and Authentication

Fig 20.8 Scope of ESP Encryption and Authentication

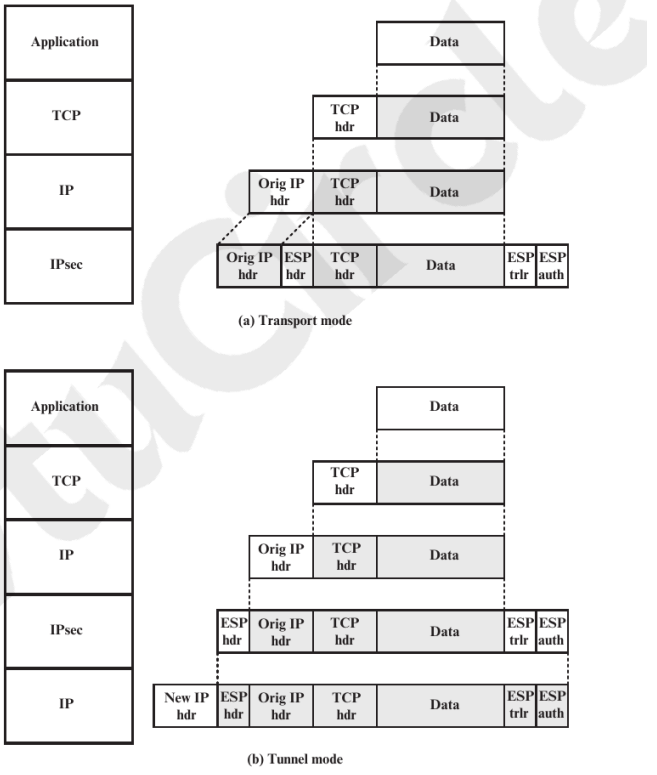


Figure 20.9 Protocol Operation for ESP

Fig 20.9 Protocol Operation for ESP – (a) transport mode, (b) tunnel mode

3.4 ESP packet format (needed for the diagrams)

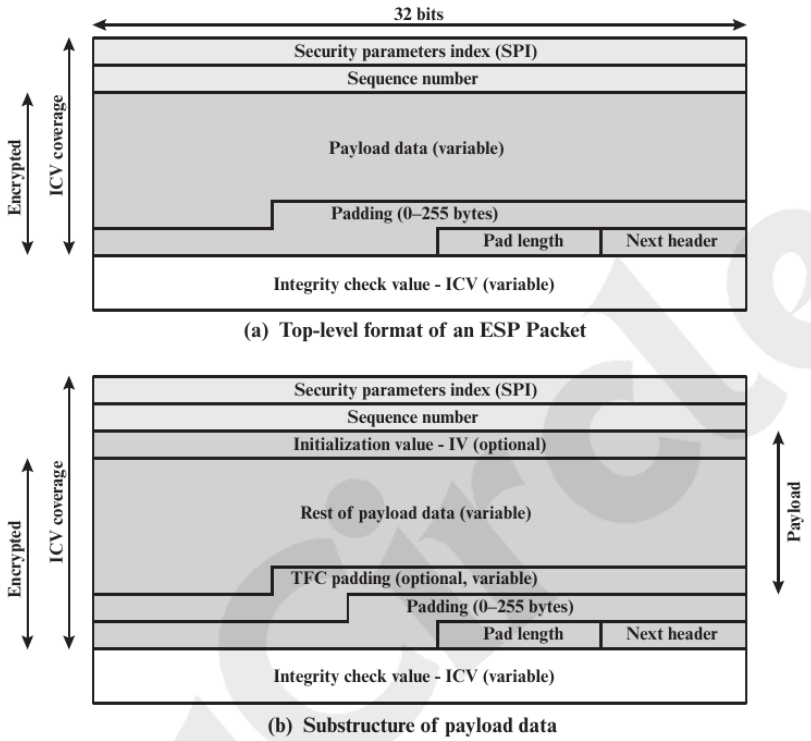


Figure 20.5 ESP Packet Format

Fig 20.5 ESP Packet Format

Field	Meaning
Security Parameters Index (32 bits)	Identifies a security association
Sequence Number (32 bits)	Monotonically increasing counter – **anti-replay**
Payload Data (variable)	Transport-level segment (transport mode) or IP packet (tunnel mode), protected by encryption
Padding (0–255 bytes)	Fits block-cipher size; right-aligns Pad Length and Next Header in a 32-bit word; extra padding gives partial traffic-flow confidentiality
Pad Length (8 bits)	Number of pad bytes immediately preceding this field
Next Header (8 bits)	Type of data in payload (first header in payload, e.g., TCP)
Integrity Check Value (variable)	ICV computed over the ESP packet minus the authentication data; optional; computed **after** encryption (quick rejection of bogus packets, parallel processing)
IV (optional) / TFC padding	IV (nonce) if required by the algorithm; TFC padding added in tunnel mode for traffic-flow confidentiality

3.5 Anti-replay service

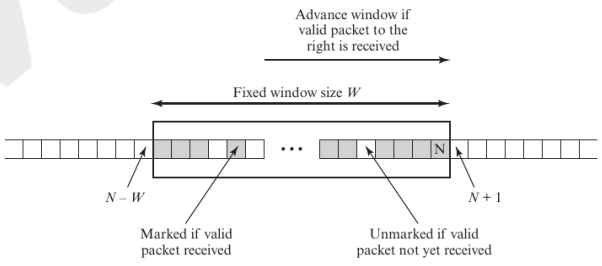


Figure 20.6 Anti-replay Mechanism

Fig 20.6 Anti-replay Mechanism

- Sender: counter initialized to **0** when SA is established; incremented for each packet (first value **1**); must not cycle past $\mathbf{2^{32} - 1}$ – then terminate the SA and negotiate a new one.
- Receiver: window of size **W** (default **64**); right edge = highest sequence number N received. (1) Packet **within window and new** → check MAC; if authentic mark the slot. (2) Packet **to the right** and new → check MAC; if authentic **advance window**. (3) Packet **to the left** of window or authentication fails → **discard** (auditable event).

Exam keywords – Transport vs Tunnel
upper-layer protocols • payload only • end-to-end host to host • ESP header before TCP • IP header not protected • traffic analysis possible • entire IP packet • new outer IP header • security gateway / VPN • inner header hidden • encapsulation • hosts need not run IPsec • counters traffic analysis • AH / ESP both modes • SPI • Fig 20.8 / 20.9

4. IP SECURITY POLICY – SECURITY ASSOCIATIONS, SAD AND SPD (Section 20.2)

IPsec policy is determined mainly by the interaction of two databases: the **Security Association Database (SAD)** and the **Security Policy Database (SPD)**.

4.1 Security Association (SA)

- A one-way logical connection between sender and receiver that affords security services to the traffic carried on it. For two-way exchange, **two SAs** are required.
- An SA is uniquely identified by **three parameters**: **Security Parameters Index (SPI)** – 32-bit value with local significance, carried in AH/ESP headers so the receiver selects the SA; **IP Destination Address** – end-user system or firewall/router; **Security Protocol Identifier** – AH or ESP.

4.2 Security Association Database (SAD) – entry parameters

Parameter	Meaning
Security Parameter Index	32-bit value chosen by the receiving end; outbound: used to build AH/ESP header; inbound: maps traffic to the SA
Sequence Number Counter	32-bit value used to generate the Sequence Number field
Sequence Counter Overflow	Flag: should overflow generate an auditable event and prevent further transmission on this SA
Anti-Replay Window	Determines whether an inbound packet is a replay
AH Information	Authentication algorithm, keys, key lifetimes, parameters
ESP Information	Encryption and authentication algorithm, keys, IVs, key lifetimes, parameters
Lifetime of this SA	Time interval or byte count after which SA must be replaced (new SPI) or terminated
IPsec Protocol Mode	Tunnel, transport or wildcard
Path MTU	Observed path maximum transmission unit and aging variables

4.3 Security Policy Database (SPD)

- Relates IP traffic to specific SAs (or no SA for traffic allowed to **bypass** IPsec). Each entry defines a subset of IP traffic and points to an SA for it.
- Each SPD entry is defined by **selectors** (IP and upper-layer protocol field values) used to filter outgoing traffic and map it to an SA: **Remote IP address**, **Local IP address** (single address, list/range or wildcard), **Next-layer protocol**, **Name** (user identifier from the OS), **Local and remote ports**.

Protocol	Local IP	Port	Remote IP	Port	Action	Comment
UDP	1.2.3.101	500	*	500	BYPASS	IKE
ICMP	1.2.3.101	*	*	*	BYPASS	Error messages
*	1.2.3.101	*	1.2.3.0/24	*	PROTECT: ESP in transport-mode	Encrypt intranet traffic
TCP	1.2.3.101	*	1.2.4.10	80	PROTECT: ESP in transport-mode	Encrypt to server
TCP	1.2.3.101	*	1.2.4.10	443	BYPASS	TLS: avoid double encryption
*	1.2.3.101	*	1.2.4.0/24	*	DISCARD	Others in DMZ
*	1.2.3.101	*	*	*	BYPASS	Internet

Table 20.2 Host SPD example. Corporate network 1.2.3.0/24; DMZ (secure LAN) 1.2.4.0/24 protected by firewalls; UDP port 500 is the designated port for IKE and bypasses IPsec processing.

5. OUTBOUND PACKETS AND INBOUND PACKETS – IP TRAFFIC PROCESSING (Section 20.2)

IPsec is executed **packet by packet**. Each **outbound** IP packet is processed by the IPsec logic **before transmission**; each **inbound** packet is processed **after reception** and before passing contents to the next higher layer (TCP/UDP).

5.1 Outbound packets (Fig 20.3)

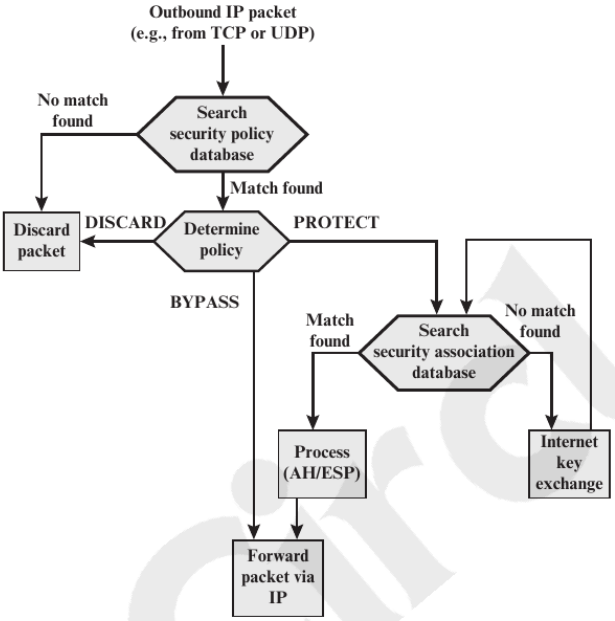


Figure 20.3 Processing Model for Outbound Packets

Fig 20.3 Processing Model for Outbound Packets

1. A block of data from a higher layer (e.g., TCP) is passed to IP and an **IP packet** (header + body) is formed. IPsec **searches the SPD** for a match.
2. If **no match** is found, the packet is **discarded** and an **error message** is generated.
3. If a match is found, the **first matching entry** decides: policy **DISCARD** → packet discarded; policy **BYPASS** → no IPsec processing, packet forwarded to the network.
4. If the policy is **PROTECT**, a search is made of the **SAD** for a matching entry. If no entry is found, **IKE is invoked** to create an SA with the appropriate keys and an entry is made in the SAD.
5. The matching SAD entry determines the processing: **encryption, authentication or both**, in **transport or tunnel mode**; the packet is then forwarded to the network.

5.2 Inbound packets (Fig 20.4)

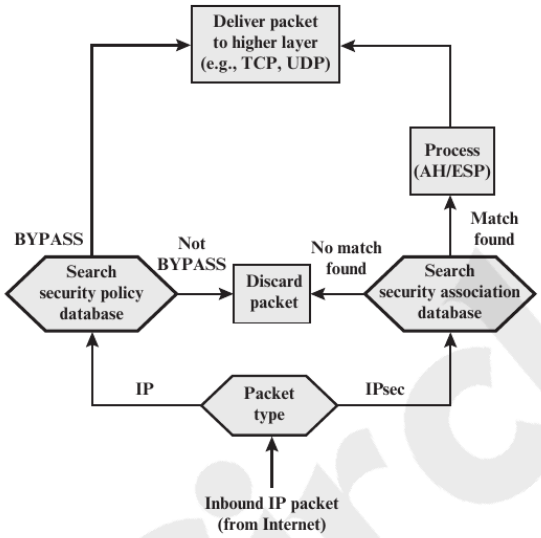


Figure 20.4 Processing Model for Inbound Packets

Fig 20.4 Processing Model for Inbound Packets

1. IPsec determines whether the packet is **unsecured** or has **ESP or AH** headers/trailers, by examining the **IP Protocol field (IPv4) or Next Header field (IPv6)**.
2. **Unsecured packet:** IPsec searches the SPD. If the first matching entry is **BYPASS** → IP header processed and stripped, body delivered to the next higher layer (TCP). If the first matching entry is **PROTECT or DISCARD**, or there is **no matching entry** → packet **discarded**.
3. **Secured packet:** IPsec searches the **SAD** (using SPI, destination address, protocol). If **no match** → **discarded**. Otherwise it applies the appropriate **ESP or AH processing**; then the IP header is processed and stripped off and the body is delivered to the next higher layer.

	Outbound	Inbound
First database searched	**SPD**	Depends on packet type: **SPD** (unsecured) / **SAD** (IPsec packet)
No match	Discard + error message	Discard
DISCARD policy	Discard	Discard
BYPASS policy	Forward without IPsec	Strip IP header, deliver to TCP/UDP
PROTECT policy	Look up SAD; if none → **IKE** creates SA; apply AH/ESP	Unsecured packet matching PROTECT → discard

Exam keywords – outbound / inbound
SPD • SAD • selectors • DISCARD • BYPASS • PROTECT • IKE invoked • create SA • SPI • AH/ESP processing • Protocol (IPv4)/Next Header (IPv6) • strip IP header • deliver to higher layer • no match → discard • packet-by-packet

6. INTERNET KEY EXCHANGE – KEY DETERMINATION ALGORITHM (Section 20.5)

IPsec key management = **determination and distribution of secret keys**. A typical requirement is **four keys** for two applications: transmit and receive pairs for both **integrity and confidentiality**. Two types are mandated:

- **Manual** – administrator manually configures each system with its own keys and those of other systems; practical for **small, relatively static** environments.
- **Automated** – **on-demand creation of keys** for SAs; suits large distributed systems with evolving configuration.

Default automated protocol = **ISAKMP/Oakley**: **Oakley Key Determination Protocol** – key exchange based on **Diffie–Hellman** with added security (generic, does not dictate formats); **ISAKMP** – framework for Internet key management with specific protocol support and formats for negotiation of security attributes (message types allowing a variety of key-exchange algorithms). In **IKEv2** the terms Oakley and ISAKMP are no longer used, but the basic functionality is the same.

6.1 Diffie–Hellman refresher

Global parameters: **q** (large prime) and **α** (primitive root of q). A picks private X_A, sends public $Y_A = \alpha^{X_A} \bmod q$. B picks X_B, sends $Y_B = \alpha^{X_B} \bmod q$. Both compute $\mathbf{K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = \alpha^{X_A X_B} \bmod q}$.

- **Attractive features:** secret keys are created **only when needed** (no long-term storage); needs **no pre-existing infrastructure** other than agreement on global parameters.
- **Weaknesses:** (1) gives **no information about the identities** of the parties; (2) subject to **man-in-the-middle attack**; (3) **computationally intensive** → vulnerable to a **clogging attack** (opponent requests many keys; victim wastes resources on modular exponentiation).

Man-in-the-middle attack on Diffie–Hellman

1. B sends his public key Y_B addressed to A.
2. Enemy E intercepts, saves Y_B, sends A a message with B's User ID but **E's public key Y_E**; similarly E sends B a message with E's public key purporting to come from A.
3. B computes K_1 from B's private key and Y_E; A computes K_2 from A's private key and Y_E; E computes K_1 (using X_E and Y_B) and K_2 (using X_E and Y_A).
4. E relays messages between A and B, changing the encipherment en route; neither A nor B knows they share communication with E.

6.2 Five features of IKE key determination

IKE key determination **retains the advantages of Diffie–Hellman while countering its weaknesses**:

1. **Cookies** to thwart **clogging attacks**.
2. Enables the two parties to negotiate a **group** (specifies the global parameters of the DH exchange).
3. Uses **nonces** to ensure against **replay attacks**.
4. Enables exchange of **Diffie–Hellman public key values**.
5. **Authenticates** the DH exchange to thwart **man-in-the-middle** attacks.

Cookie exchange (anti-clogging)

- In a clogging attack the opponent **forges the source address** of a legitimate user and sends a public DH key; the victim performs modular exponentiation. Repeated messages **clog** the victim.
- **Cookie exchange:** each side sends a **pseudorandom number (cookie)** in the initial message, which the other acknowledges; the acknowledgment must be repeated in the first message of the DH exchange. If the source address was forged, the opponent gets **no answer** – so it can only force the victim to generate acknowledgments, not to perform DH calculation.

Cookie requirements: (1) must **depend on the specific parties** (prevents an attacker reusing a cookie from a real IP/UDP port against random addresses); (2) only the **issuing entity** can generate cookies it accepts – uses **local secret information** that cannot be deduced from a cookie, so it need not save cookies; (3) generation and verification must be **fast**. **Recommended method:** a fast hash (e.g., **MD5**) over the **IP source and destination addresses, UDP source and destination ports and a locally generated secret value**.

Groups (Diffie–Hellman)

- Modular exponentiation with **768-bit** modulus: $q = 2^{768} - 2^{704} - 1 + 2^{64} \times (\lfloor 2^{638} \times \pi \rfloor + 149686)$, $\alpha = 2$.
- Modular exponentiation with **1024-bit** modulus: $q = 2^{1024} - 2^{960} - 1 + 2^{64} \times (\lfloor 2^{894} \times \pi \rfloor + 129093)$, $\alpha = 2$.
- Modular exponentiation with **1536-bit** modulus (parameters to be determined).
- **Elliptic curve group over 2^{155}** (generator X = 7B, Y = 1C8; A = 0, Y = 7338F) and **over 2^{185}** (generator X = 18, Y = D; A = 0, Y = 1EE9). First three = classic DH; last two = **elliptic-curve analog** of DH.

Nonces and authentication methods

- **Nonce** = locally generated pseudorandom number; appears in responses and is encrypted during certain portions of the exchange – protects against **replay**.
- **Three authentication methods: Digital signatures** (sign a mutually obtainable hash over IDs and nonces with private key); **Public-key encryption** (encrypt parameters such as IDs and nonces); **Symmetric-key encryption** (key derived by out-of-band mechanism encrypts exchange parameters).

6.3 IKEv2 exchanges (Fig 20.11)

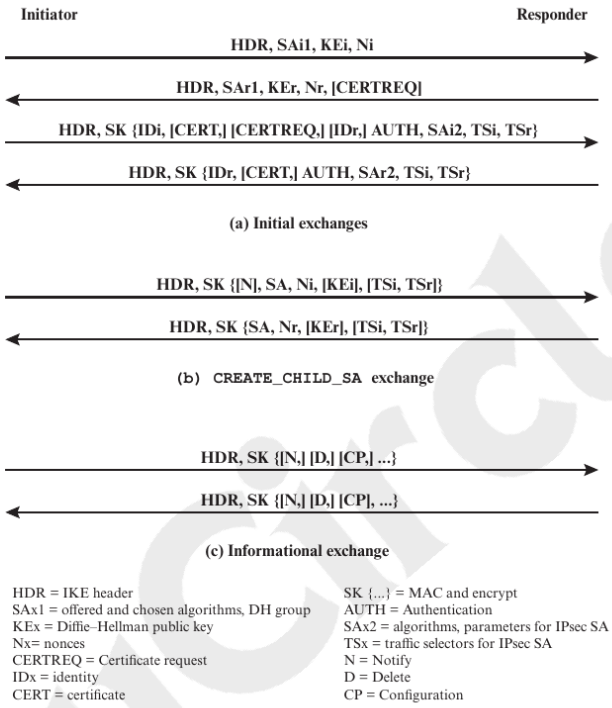


Figure 20.11 IKEv2 Exchanges

Fig 20.11 IKEv2 Exchanges

- Messages are exchanged in **pairs**. The first two pairs are the **initial exchanges** (a).
- **1st exchange:** peers exchange cryptographic algorithms and security parameters they will use, plus **nonces** and **DH values** (HDR, SA, KE, N). Result: a special **IKE SA** defines a secure channel for all later IKE messages (protected by **encryption and message authentication**).
- **2nd exchange:** the two parties **authenticate** one another (ID, CERT, AUTH) and set up the **first IPsec SA** (child SA) placed in the SADB, used to protect ordinary (non-IKE) traffic. So **four messages** establish the first SA for general use.
- **CREATE_CHILD_SA exchange (b):** establishes **further SAs** for protecting traffic.
- **Informational exchange (c):** exchanges **management information, IKEv2 error messages and other notifications** (N, D, CP).

7. IKE HEADER FORMAT AND PAYLOADS (Section 20.5)

IKE defines procedures and packet formats to **establish, negotiate, modify and delete SAs** and payloads for exchanging key-generation and authentication data, independent of the specific key-exchange protocol, encryption algorithm and authentication mechanism. An IKE message = **IKE header followed by one or more payloads**, carried over a transport protocol – implementations **must support UDP**.

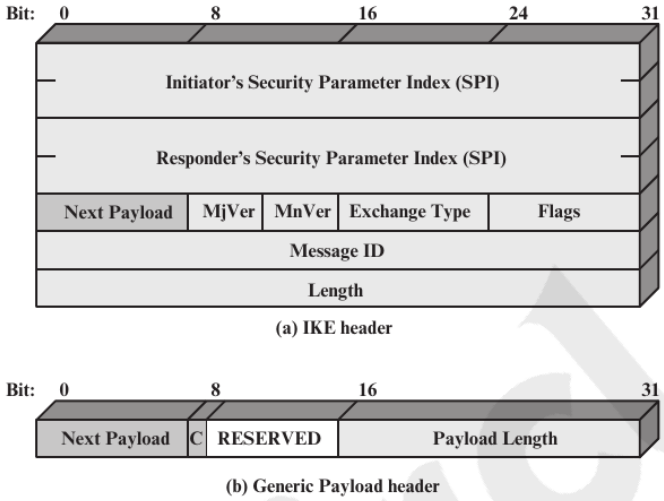


Figure 20.12 IKE Formats

Fig 20.12 IKE Formats – (a) IKE header (b) Generic payload header

7.1 IKE header fields (Fig 20.12a)

Field	Size	Meaning
Initiator SPI	64 bits	Chosen by the initiator to identify a unique IKE SA
Responder SPI	64 bits	Chosen by the responder to identify a unique IKE SA
Next Payload	8 bits	Type of the **first payload** in the message
Major Version	4 bits	Major version of IKE in use
Minor Version	4 bits	Minor version in use
Exchange Type	8 bits	Type of exchange (initial, CREATE_CHILD_SA, informational)
Flags	8 bits	Options for this exchange; three bits defined: **Initiator** bit (packet sent by SA initiator), **Version** bit (transmitter can use a higher major version), **Response** bit (response to a message with the same message ID)
Message ID	32 bits	Controls **retransmission of lost packets** and matching of requests and responses
Length	32 bits	Length of total message (header + all payloads) in octets

7.2 Generic payload header (Fig 20.12b)

- **Next Payload** – value **0 if this is the last payload**; otherwise the type of the next payload.
- **C (critical) bit** – 0: recipient may **skip** this payload if it does not understand the type code in the Next Payload field of the previous payload; 1: recipient must **reject the entire message** if it does not understand the payload type.
- **RESERVED**; **Payload Length** – length in octets of this payload including the generic header.

7.3 IKE payload types (Table 20.3)

Type	Parameters / purpose
Security Association (SA)	Proposals – begins establishment of an SA. Hierarchy: **Proposal** (number, protocol ID AH/ESP/IKE, number of transforms) → **Transform** (cryptographic algorithms for the protocol) → **Attribute** (e.g., key length)
Key Exchange	DH group #, key-exchange data (Oakley, DH, RSA-based exchange used by PGP)
Identification	ID type, ID data (identity of peers; usually IPv4/IPv6 address)
Certificate	Cert encoding, certificate data (PKCS #7 wrapped X.509, PGP, DNS signed key, X.509 signature/key-exchange, Kerberos tokens, CRL, ARL, SPKI)
Certificate Request	Cert encoding, certification authority – request the other entity's certificate
Authentication	Auth method, authentication data (RSA digital signature, shared-key message integrity code, DSS digital signature)
Nonce	Nonce data – liveness and replay protection
Notify	Protocol-ID, SPI size, notify message type, SPI, notification data – **error or status** information

Type	Parameters / purpose
Delete	Protocol-ID, SPI size, # of SPIs, SPI(s) – SAs the sender has deleted
Vendor ID	Vendor-defined constant to recognise remote implementations; lets a vendor experiment while keeping backward compatibility
Traffic Selector	Number of TSs, traffic selectors – identify packet flows for IPsec processing
Encrypted	IV, encrypted IKE payloads, padding, pad length, ICV – contains other payloads in encrypted form (similar to ESP)
Configuration	CFG type, configuration attributes – exchange configuration information between peers
EAP	EAP message – lets IKE SAs be authenticated using the **Extensible Authentication Protocol**

Notify messages – *Error:* Unsupported Critical Payload, Invalid IKE SPI, Invalid Major Version, Invalid Syntax, Invalid Payload Type, Invalid Message ID, Invalid SPI, No Proposal Chosen, Invalid KE Payload, **Authentication Failed**, Single Pair Required, No Additional SAs, Internal Address Failure, Failed CP Required, TS Unacceptable, Invalid Selectors. *Status:* Initial Contact, Set Window Size, Additional TS Possible, IPCOMP Supported, NAT Detection Source/Destination IP, **Cookie**, Use Transport Mode, HTTP Cert Lookup Supported, Rekey SA, ESP TFC Padding Not Supported, Non First Fragments Also.

Exam keywords – IKE
manual vs automated • ISAKMP/Oakley • IKEv2 • Diffie–Hellman • clogging attack • cookies (MD5 hash) • groups (768, 1024, 1536, ECC 2^155, 2^185) • nonce • authentication: digital signature / public-key / symmetric • man-in-the-middle • initial exchanges • IKE SA • CREATE_CHILD_SA • informational exchange • UDP port 500 • Initiator SPI 64-bit • Responder SPI • Next Payload • MjVer MnVer • Exchange Type • Flags (initiator, version, response) • Message ID • Length • generic payload header • critical bit • proposal-transform-attribute

LAST-MINUTE REVISION SHEET – MODULE 5

Topic	One-line recall
DKIM	Domain signs mail with private key; verifier gets public key from DNS; RFC 6376; header tags v a c d s h bh b; canonicalization simple/relaxed; RSA-SHA256; transparent to user.
IPsec applications	Branch office VPN, remote access, extranet/intranet, e-commerce.
IPsec benefits	Firewall/router perimeter security • bypass-resistant • below transport layer (transparent to apps) • transparent to users • individual-user security.
Transport mode	Host-to-host; protects payload; IP header not encrypted; traffic analysis possible.
Tunnel mode	Gateway-based; entire inner packet encrypted; new outer IP header; hosts need no IPsec; VPN.
Outbound	SPD search → (no match: discard) → DISCARD / BYPASS / PROTECT → SAD search → (none: IKE) → AH/ESP → forward.
Inbound	IP or IPsec? → IP: SPD (only BYPASS delivers) • IPsec: SAD (no match discard) → AH/ESP processing → deliver.
IKE	DH + cookies (anti-clogging) + groups + nonces (anti-replay) + authentication (anti-MITM); IKEv2: initial exchanges (4 messages), CREATE_CHILD_SA, informational; header = Initiator SPI, Responder SPI, Next Payload, MjVer, MnVer, Exchange Type, Flags, Message ID, Length.